

EE6407: Genetic Algorithms and Machine Learning

Quiz 1 Review Notes — Evolutionary Computing Part 1 (Weeks 1–4)

Living revision notes

Updated 24 August 2026

Sources and scope. This note covers the complete evolutionary-computing Part 1 material in `../resources/lecture-01-evolutionary-computing-part-1.pdf` (36 PDF pages, 72 numbered slides), supported by the Week 1 and Week 2 class transcripts in `../resources/week-01-transcript.txt` and `../resources/week-02-transcript.txt`. The specialised N -queens method is based on Gwee and Lim, “An evolution search algorithm for solving N -queen problems,” supplied as `../resources/n-queens-evolution-search-paper.pdf` (especially article pp. 1–4). The latter is a particular evolutionary search algorithm, not the only valid genetic algorithm for the problem.

Assessment facts visible in the supplied lecture slides. Continuous assessment contributes 40% and the three-hour final examination contributes 60%. For the first four-week part, one closed-book written quiz contributes 10% and is scheduled for the last hour of Week 4. Exact date, time, room allocation, and any later changes should be checked in NTULearn.

Contents

1	Why Evolutionary Computing?	2
1.1	Historical landmarks	2
2	Classifying Problems Before Solving Them	2
2.1	The input–model–output view	2
2.2	Search spaces and problem solvers	3
2.3	Optimisation and constraint satisfaction	3
2.4	P, NP, NP-complete, and NP-hard	3
3	From Natural Evolution to an Algorithm	4
3.1	Genotype, phenotype, genes, and alleles	4
4	The General Evolutionary Algorithm	4
4.1	Fitness and population	5
4.2	Parent and survivor selection	5
4.3	Variation	5
4.4	Initialisation and termination	6
5	Worked Model: The N-Queens Problem	6
5.1	Fitness formulations	6
5.2	Representation-aware operators	7
5.3	The specialised two-stage evolutionary search	7
6	Worked Model: Simple GA for $f(x) = x^2$	7

7	EAs in Optimisation Context	8
7.1	Domain knowledge and no free lunch	8
8	Quiz 1 Rapid Review	8
8.1	Definitions and formulas to reproduce	9
8.2	Algorithm steps and calculation checks	9
8.3	Likely traps	9
8.4	Final recall checklist	9
9	Common Pitfalls	10
10	Revision Checklist	10

1 Why Evolutionary Computing?

Evolutionary computing (EC) turns a simplified metaphor of biological evolution into a family of computational search methods. A population of candidate solutions competes under a task-specific fitness measure. Selected candidates create new candidates through variation, and the process repeats. Genetic algorithms (GAs) are one historically important member of this wider family.

The attraction is practical: many engineering, design, scheduling, and learning problems have a huge or irregular search space, mixed discrete and continuous decisions, non-differentiable objectives, or many local optima. An evolutionary algorithm (EA) does not require gradients and can search with multiple candidates at once. This flexibility does *not* imply a guarantee of finding the global optimum.

1.1 Historical landmarks

The lecture highlights Turing’s proposal of genetical or evolutionary search (1948), Bremermann’s optimisation by evolution and recombination (1962), Rechenberg’s evolution strategies (1964), Fogel, Owens, and Walsh’s evolutionary programming (1965), Holland’s genetic algorithms (1975), and Koza’s genetic programming (1992). These traditions originally emphasised different representations, but modern practice starts from the problem and chooses a suitable representation and operators.

2 Classifying Problems Before Solving Them

The purpose of classification is not taxonomy for its own sake. It exposes what is unknown, how large the search is, what counts as a good answer, and what computational difficulty should be expected.

2.1 The input–model–output view

Represent a system abstractly by

$$y = M(x),$$

where x is the input, M the model, and y the output. Holding two parts fixed and seeking the third gives three different problem types.

Unknown	Given	Problem type	Typical question
x	M, y	optimisation	Which design/input gives the desired output?
M	paired x, y data	modelling	Which mapping explains or predicts the data?
y	x, M	simulation	What happens under this input and model?

Examples include university timetabling, satellite structural design, the travelling-salesman problem (optimisation), credit-risk prediction and voice control (modelling), and weather or policy “what-if” studies (simulation). A modelling task can often be converted into optimisation by parameterising M_θ and minimising a prediction loss over θ .

2.2 Search spaces and problem solvers

The *search space* S is the set of all candidate objects under consideration; the feasible set $\mathcal{F} \subseteq S$ contains candidates that satisfy every hard constraint. A problem definition describes S , $\mathcal{F}$, and the desired quality. A problem solver describes how to move through or sample S . Keeping these notions separate prevents a common mistake: a large search space is a property of the formulation, while slow progress may also result from an unsuitable algorithm or representation.

For a tour through n labelled cities, the naive number of orderings is $n!$ (or $(n-1)!/2$ if rotations and reversal are identified for a symmetric tour). Exhaustive enumeration therefore becomes impractical very quickly.

2.3 Optimisation and constraint satisfaction

An objective function $f : S \rightarrow \mathbb{R}$ ranks candidate quality. A constraint is a condition such as $g_j(x) \leq 0$ or $h_k(x) = 0$.

- A *free optimisation problem* has an objective but no explicit constraints.
- A *constraint-satisfaction problem* (CSP) asks only for a feasible point.
- A *constrained optimisation problem* (COP) asks for the best feasible point.

A CSP can be transformed into optimisation by defining a violation measure, for example

$$V(x) = \sum_j \max\{0, g_j(x)\} + \sum_k |h_k(x)|,$$

and minimising V . A COP can use a penalty objective

$$F_\lambda(x) = f(x) + \lambda V(x)$$

for a minimisation problem. The penalty weight λ must be chosen with care: too small allows attractive but infeasible candidates; too large can flatten useful distinctions among infeasible candidates.

2.4 P, NP, NP-complete, and NP-hard

Let the input size be n , measured by the number of bits or symbols required to encode an instance. Running time is usually discussed in the worst case as a function of n .

- **P**: decision problems solvable in polynomial time.

- **NP**: decision problems whose proposed yes-certificates can be verified in polynomial time. Thus $\mathbf{P} \subseteq \mathbf{NP}$.
- **NP-complete**: problems in **NP** to which every problem in **NP** can be reduced in polynomial time.
- **NP-hard**: problems at least as hard as the **NP-complete** problems; they need not themselves be decision problems or belong to **NP**.

It remains unknown whether $\mathbf{P} = \mathbf{NP}$. Metaheuristics such as EAs are therefore used to obtain good solutions within a practical budget, especially when exact solution is too expensive. Their use does not change the complexity class of the underlying problem.

3 From Natural Evolution to an Algorithm

The simplified Darwinian metaphor has four essential ideas:

1. resources are finite, so selection is unavoidable;
2. individuals differ in phenotype;
3. some differences affect reproductive success and are heritable;
4. random variation continually introduces diversity.

Individuals are the units on which selection acts, but the population is the unit that evolves. Computationally, fitness is an explicitly designed score, unlike natural fitness, which is inferred indirectly from reproductive success.

3.1 Genotype, phenotype, genes, and alleles

The *phenotype* is a solution in the original problem domain. The *genotype* is the manipulable encoding used by the EA. A chromosome is composed of genes at loci, and each gene takes an allele value.

Let e be an encoder and d a decoder:

$$e : \mathcal{F} \rightarrow G, \quad d : G \rightarrow S.$$

The decoder must give an unambiguous candidate for every admissible genotype. For global search, every feasible phenotype should be reachable from at least one genotype. Encoding need not be one-to-one, but redundancy can bias the search because phenotypes with many encodings are sampled more often.

Representation principle. Operators must preserve or sensibly repair the chosen representation. Bit flip suits binary strings; Gaussian perturbation suits real vectors; swap or insertion mutations suit permutations. Using ordinary one-point crossover on a permutation may duplicate some values and omit others, producing an invalid solution.

4 The General Evolutionary Algorithm

An EA is a stochastic, population-based, generate-and-test algorithm. One generic form is:

1. initialise a population P_0 of candidate genotypes;

2. decode and evaluate every candidate;
3. while the termination condition is false:
 - (a) select parents from P_t ;
 - (b) recombine selected parents;
 - (c) mutate the resulting offspring;
 - (d) evaluate the offspring;
 - (e) select survivors to form P_{t+1} .

The process balances two opposing forces:

$$\underbrace{\text{mutation + recombination}}_{\text{novelty/diversity}} \quad \text{against} \quad \underbrace{\text{parent + survivor selection}}_{\text{quality pressure}}.$$

Excessive selection pressure can cause premature convergence; insufficient pressure wastes evaluations on poor regions. Diversity is therefore not an end in itself but a resource that keeps alternatives available.

4.1 Fitness and population

The fitness or evaluation function maps each phenotype to a scalar used for comparison. It must reflect the actual task, including constraint handling, and discriminate usefully among candidates. If the natural objective is a cost $c(x)$ to minimise, it is often safer to keep a minimisation-aware selection method or use a monotone transformation rather than the singular $1/c(x)$ when $c(x)$ may be zero.

A population is a multiset, so duplicate genotypes are allowed. Diversity can refer to genotype, phenotype, or fitness diversity; these are different. For example, several genotypes may decode to the same phenotype, and distinct phenotypes may have equal fitness.

4.2 Parent and survivor selection

Selection favours quality without normally making the outcome certain. In fitness-proportionate (roulette-wheel) selection, assuming non-negative fitness,

$$\Pr(i \text{ selected}) = \frac{f_i}{\sum_{j=1}^{\mu} f_j}.$$

Thus candidates A, B, C with fitness 3, 1, 2 receive probabilities $1/2, 1/6, 1/3$. Scaling matters: adding a constant to all fitness values changes these probabilities even though the ranking is unchanged. Rank or tournament selection avoids this particular sensitivity.

Survivor selection controls replacement. Common choices include deleting all parents after producing μ offspring, keeping the best μ among parents and offspring, or using *elitism* to guarantee that a small number of the best candidates survive. Strong elitism preserves progress but can reduce diversity.

4.3 Variation

Mutation is unary and makes a random change to one genotype. It introduces or preserves diversity and, when the mutation graph connects the whole genotype space, makes every solution reachable in principle.

Recombination has arity greater than one, usually two, and merges information from parents. A child is not guaranteed to be better than either parent; the hope is that useful partial structures can occasionally be combined.

4.4 Initialisation and termination

Random initialisation should cover the admissible allele values without unintended bias. Known solutions or domain heuristics may seed the population, but excessive seeding can suppress exploration.

Typical stopping rules are a maximum generation or evaluation count, reaching a target fitness, no meaningful improvement for several generations, or diversity falling below a threshold. Stopping because the population is unchanged is evidence of stagnation, not proof of global optimality.

5 Worked Model: The N -Queens Problem

Place N queens on an $N \times N$ board so that no two share a row, column, or diagonal. With exactly one queen assigned to each row, encode a candidate as a permutation

$$q = (q_1, q_2, \dots, q_N),$$

where $q_i \in \{1, \dots, N\}$ is the column of the queen in row i . The permutation property gives

$$q_i \neq q_j \quad (i \neq j),$$

so row and column conflicts are impossible by construction. Only diagonal conflicts remain:

$$|q_i - q_j| \neq |i - j| \quad \text{for every } i \neq j.$$

This illustrates a major design lesson: a good representation can remove hard constraints from the search rather than repeatedly penalising them.

5.1 Fitness formulations

Define a conflict indicator for a pair $i < j$:

$$a_{ij}(q) = \begin{cases} 1, & |q_i - q_j| = |i - j|, \\ 0, & \text{otherwise.} \end{cases}$$

The total number of attacking pairs is

$$C(q) = \sum_{1 \leq i < j \leq N} a_{ij}(q).$$

We may minimise $C(q)$, or maximise

$$f_{\text{pairs}}(q) = \binom{N}{2} - C(q).$$

The optimum satisfies $C(q) = 0$.

The supplied paper instead gives each queen a binary gene fitness

$$c_i(q) = \begin{cases} 1, & \text{queen } i \text{ conflicts with no other queen,} \\ 0, & \text{otherwise,} \end{cases} \quad F(q) = \sum_{i=1}^N c_i(q).$$

Here $F(q) = N$ exactly at a solution. This differs from counting non-attacking pairs: both identify the same optima, but they shape the intermediate fitness landscape differently.

5.2 Representation-aware operators

A valid mutation swaps the alleles at two randomly selected loci. The result remains a permutation. A valid crossover can copy a prefix from one parent, then scan the other parent cyclically and append values not already present. This is an order-preserving construction: one parent supplies explicit values, while the other supplies relative order.

One lecture example chooses five candidates at random and uses the best two as parents. A child may replace the first lower-fitness individual in a population sorted by decreasing fitness. These are design choices, not fixed requirements of all GAs.

5.3 The specialised two-stage evolutionary search

Gwee and Lim reduce the unrestricted N^N configurations to the $N!$ permutation space. Their article reports that solution density still falls sharply: for $N = 7$ there are about 7936 solutions per million permutations, whereas for $N = 12$ there are about 29 per million (article p. 1).

Their algorithm has two stages:

1. **Selection and reproduction.** Evaluate and sort the population; retain the fitter half and create related offspring to replace the weaker half. A shift-by-one reproduction maps, for example, $(1, 3, 5, 2, 4)$ to $(2, 4, 1, 3, 5)$.
2. **Selection and mutation.** Identify *weak genes*, namely queens involved in conflicts, and preferentially mutate them. Stop when $F(q) = N$ or the generation limit is reached.

Mutating only conflicting queens can oscillate between local peaks. The paper's remedy is to label one or more otherwise fit genes as *pseudo-weak*; mutation may then disturb a larger structure and escape the cycle. This is a concrete example of injecting exploration when a domain-specific operator has become too restrictive. The reported simulations found instances up to $N = 2000$ on a personal computer and, for the tested settings, first solutions within 30 generations (article p. 4). These are empirical results for that implementation, not universal complexity bounds.

6 Worked Model: Simple GA for $f(x) = x^2$

Suppose x is an integer in $[0, 31]$ and the goal is to maximise x^2 . Five bits encode every feasible value:

$$g = b_4b_3b_2b_1b_0,$$
$$x = d(g) = \sum_{k=0}^4 2^k b_k, \quad f(g) = d(g)^2.$$

A hand simulation proceeds as follows:

1. sample a population of five-bit strings;
2. decode each string and compute x^2 ;
3. select a mating pool using fitness proportions;
4. pair parents and sample a one-point crossover location from the four gaps between bits;
5. independently flip each bit with mutation probability p_m ;
6. evaluate the new population and repeat.

For population size 4, the population contains 20 bits. With $p_m = 0.001$, the expected number of mutated bits per generation is only $20(0.001) = 0.02$. This does *not* mean exactly 0.02 bits mutate; the actual count is random and is usually zero in a particular generation. The probability of no mutation is

$$(1 - p_m)^{20} = 0.999^{20} \approx 0.9802.$$

This distinction between expectation and realised events is important when interpreting stochastic algorithms.

7 EAs in Optimisation Context

For a fixed feasible set S , global optimisation seeks

$$x^* \in \arg \max_{x \in S} f(x) \quad \text{or} \quad x^* \in \arg \min_{x \in S} f(x).$$

Exact deterministic methods may guarantee x^* but require super-polynomial time in difficult cases. Heuristics decide which candidate to test next and often find useful answers quickly, but ordinarily provide no certificate of global optimality.

Hill climbing uses a neighbourhood and often terminates at a local optimum. EAs differ through a population, stochastic selection, and multiple variation operators, especially recombination. Their effective neighbourhood is operator-dependent: mutation connects nearby genotypes, while crossover can jump between regions defined by parental material.

7.1 Domain knowledge and no free lunch

Adding domain knowledge through representations, repair rules, seeded solutions, or specialised operators can improve performance on the intended problem class. It simultaneously introduces bias and may hurt performance on other classes. The lecture’s “no free lunch” message is therefore not that all algorithms are equally useful on a chosen engineering problem. It is that, averaged uniformly over all possible objective functions under the theorem’s assumptions, gains on some problems are offset by losses on others. The practical conclusion is to match algorithm bias to the problem and test empirically on representative instances.

8 Quiz 1 Rapid Review

Scope boundary. Prepare all of Part 1 in the Lecture 1 Part 1 deck, PDF pages 1–36: classification, complexity, EC foundations, the EA loop and components, 8-queens, the simple GA, EA behaviour, and optimisation context. Historical questions support the patterns of classification, representation, encoding, fitness, and hand calculations only. Differential evolution, PSO, advanced crossover families, schema theory, niching, multi-objective optimisation, and machine learning are not part of this Quiz 1 pack.

8.1 Definitions and formulas to reproduce

x unknown $\Rightarrow$ optimisation,

M unknown $\Rightarrow$ modelling,

y unknown $\Rightarrow$ simulation,

$$p_i = \frac{f_i}{\sum_j f_j},$$

$$|q_i - q_j| = |i - j|,$$

$$f(q) = \binom{N}{2} - C(q),$$

$$z = \sum_{k=0}^{L-1} 2^k b_k,$$

$$x = a + \frac{z}{2^L - 1}(b - a),$$

$$\mathbb{E}[\text{flips}] = NLp_m,$$

$$\Pr(\text{no flips}) = (1 - p_m)^{NL}.$$

P means polynomial-time solvable; **NP** means polynomial-time verifiable yes-certificates; NP-complete means both in NP and NP-hard; NP-hard does not require membership in NP or even a decision-problem form. A phenotype is the domain solution, while a genotype is the encoding manipulated by the EA.

8.2 Algorithm steps and calculation checks

initialise $\rightarrow$ evaluate $\rightarrow$ select parents $\rightarrow$ recombine $\rightarrow$ mutate
 $\rightarrow$ evaluate offspring $\rightarrow$ select survivors $\rightarrow$ stop/repeat.

1. Write the objective direction and larger-is-better fitness convention before calculating selection probabilities.
2. Decode every chromosome after crossover and mutation; recompute fitness rather than carrying an old value forward.
3. Check roulette probabilities against the correct population and verify that they sum to 1 up to rounding.
4. For permutation encodings, check for duplicates and omissions after every operator. Swap mutation preserves validity; ordinary one-point crossover generally does not.
5. For N -queens, inspect each pair once with $i < j$ and use $\binom{N}{2}$ as the maximum fitness.
6. Distinguish an expected mutation count from the realised random number of flips.

8.3 Likely traps

NP does not mean non-polynomial; selection favours but does not guarantee; parent selection and survivor selection answer different questions; high fitness pressure can destroy diversity; convergence is not a proof of global optimality; a local optimum depends on the chosen neighbourhood; and no-free-lunch does not say that algorithm choice is irrelevant on a particular problem.

8.4 Final recall checklist

Without looking, can you:

- classify all three $y = M(x)$ cases and identify S , $\mathcal{F}$, and f ;
- draw the complete EA loop and state the role of each component;
- state the P/NP/NP-complete/NP-hard relationships without overclaiming;
- define genotype, phenotype, chromosome, gene, and allele;
- score a queen permutation and explain operator validity;
- decode a binary string, calculate roulette probabilities, perform one crossover and mutation, and verify the resulting fitnesses; and
- explain exploration, exploitation, local versus global optimality, domain bias, and the practical no-free-lunch message?

9 Common Pitfalls

1. **Treating GA and EC as exact synonyms.** GA is a member of the broader EC family, even if the lecture sometimes uses the terms informally and interchangeably.
2. **Confusing phenotype with genotype.** Fitness belongs to the decoded problem solution, even when it is computed directly from a convenient encoding.
3. **Using invalid operators.** Operators designed for bit strings need not preserve permutations or constraints.
4. **Assuming fitness always improves.** Variation can produce worse children; improvement arises statistically through selection.
5. **Equating convergence with proof.** A stagnant population may be trapped locally.
6. **Quoting empirical performance as a guarantee.** Results such as “under 30 generations” depend on the paper’s instances, parameters, operators, and implementation.
7. **Ignoring fitness scaling.** Roulette-wheel probabilities can become unstable with negative values or a single dominant value.

10 Revision Checklist

You should be able to:

- classify a task using the input–model–output view;
- distinguish a search space, feasible set, objective, and solver;
- state the meanings and relationships of P, NP, NP-complete, and NP-hard without claiming that $P \neq NP$ is proved;
- draw the EA loop and explain the roles of representation, evaluation, population, selection, recombination, and mutation;
- explain why variation promotes novelty while selection promotes quality;
- compute roulette-wheel probabilities and discuss their sensitivity;
- formulate N -queens as a permutation and derive a conflict-based fitness;
- choose mutation and crossover operators that preserve a permutation;
- explain weak and pseudo-weak genes in the supplied N -queens paper;
- distinguish a good empirical result from a global-optimality or runtime guarantee.